



HifMsgDataCollectionLib

使用说明

文档页数：35
文档版本号：1.1
发布时间：2026-06-24

版本历史

版本	版本描述	日期
1.1	1、新增支持消息 Id 0xC2 上报的 datacube 数据解析； 2、新增支持消息 Id 0xC3 上报的点云数据解析； 3、新增下发配置文件、下发数据的接口； 4、新增唤醒设备、停止雷达、启动雷达接口； 5、数据类型 dataType=0 重新定义为 psic_debug_data，废弃 pointCloudRawDataOrStringData 属性，改用 payloadData 属性替代； 6、废弃 PointCloudConversion 转换接口，改用 PsicDebugDataConversion 替代； 7、更新 Python 使用示例； 8、增加常见问题的排查方向，详情见章节 7。	2025-06-24
1.0	创建文档	2025-11-04

目录

版本历史..... ii

目录..... iii

图片目录..... v

表格目录..... vi

1 概述..... 7

2 软硬件环境搭建..... 8

 2.1 硬件环境搭建..... 8

 2.2 软件环境搭建..... 8

3 使用流程介绍..... 9

 3.1 API 接口调用主流程..... 9

 3.2 数据采集流程..... 10

 3.3 发送配置文件或发送自定义数据流程..... 11

 3.4 数据采集过程中发送配置流程..... 12

 3.5 数据转换接口关系说明..... 13

4 API 接口说明..... 14

 4.1 API 接口说明..... 14

 4.2 错误码说明..... 17

5 相关数据类型..... 18

 5.1 数据采集回调事件类型..... 18

 5.1.1 Uint32[] datacubeRawData 数据格式..... 19

 5.1.2 当 dataType = 0 or 1 时, byte[] payloadData 数据格式..... 19

 5.2 数据转换返回 PsicDebugData 类型..... 21

 5.3 数据转换返回 PointCloudData 类型..... 22

 5.4 数据转换返回 DataCubeData 类型..... 22

6 使用示例 (Python) 25

 6.1 主流程..... 25

 6.2 Python 环境的准备..... 26

 6.3 DllLoader 类..... 26

 6.4 DataCollection 类..... 27

 6.4.1 初始化函数..... 27

 6.4.2 Python 回调函数..... 28

 6.4.3 数据转换线程函数..... 28

 6.4.4 下发配置文件示例..... 29

6.4.5 自定义数据发送示例..... 29

6.4.6 数据解析输出示例..... 29

6.5 数据解析说明..... 30

6.5.1 dataType = 0: psic_debug 解析.....30

6.5.2 dataType = 1: 字符串数据解析..... 30

6.5.3 dataType = 2: datacube_0xC1 解析..... 31

6.5.4 dataType = 3: datacube_0xC2 解析..... 31

6.5.5 dataType = 4: point_cloud 解析..... 32

7 常见问题..... 33

7.1 为什么打开串口失败..... 33

7.2 为什么打开 SPI 设备失败..... 33

7.3 为什么没有接收到数据或打印数据不完整..... 33

7.4 发送配置异常排查方向..... 34

图片目录

图 3-1	API 接口调用主流程示意图.....	9
图 3-2	数据采集流程示意图.....	10
图 3-3	发送配置流程示意图.....	11
图 3-4	数据采集过程中发送配置的流程示意图.....	12
图 5-1	UInt32[] datacubeRawData 数据格式.....	19
图 5-2	dataType = 0 时, byte[] pointCloudRawDataOrStringData 数据格式.....	20
图 5-3	dataType = 1 时, byte[] pointCloudRawDataOrStringData 数据格式.....	21

表格目录

表 3-1 不同类型数据对应的转换接口说明表.....13

表 4-1 API 接口说明.....14

表 4-2 错误码说明..... 17

表 5-1 dataType = 0 时，byte[] pointCloudRawDataOrStringData 参数定义..... 20

表 6-1 DllLoader 类说明.....26

表 6-2 DataCollection 类说明..... 27

表 7-1 串口打开异常..... 33

表 7-2 SPI 打开异常..... 33

表 7-3 没接收到数据..... 33

表 7-4 配置异常排查方向..... 34

1 概述

本文档旨在介绍开发者如何在自定义的上位机中，实现对 RS6x_7x 系列毫米波雷达传感器芯片上报的数据进行采集并解析得到真正有效的数据。

开发者通过本公司提供的上位机 HifMsgDataCollection.dll 库文件，可以实现在自定义的上位机中集成 Datacube 数据、点云数据、调试数据的采集功能、数据转换功能以及下发配置等功能。

HifMsgDataCollection.dll 文件提供了两种通信方式（UART、SPI）的数据接收方式，开发者可以根据应用场景选择合适的方式接收 RS6x_7x 系列毫米波雷达传感器芯片上报的数据。

注意：HifMsgDataCollection.dll 的 SPI 通信采用第三方转接芯片（CH347T 转 USB 芯片），若使用 SPI 方式进行数据采集需要先安装 CH347T 转接芯片的 Window 驱动。

2 软硬件环境搭建

2.1 硬件环境搭建

开发者可以参考“RS6x_7X 毫米波雷达开发_快速入门”进行硬件环境搭建。

2.2 软件环境搭建

开发者可以参考“RS6x_7X 毫米波雷达开发_快速入门”进行软件环境的搭建，并运行工程上报数据。

3 使用流程介绍

3.1 API 接口调用主流程

如下图所示,为用户在自定义的上位机中调用 HifMsgDataCollectionLib.dll 提供的 API 接口实现数据采集、数据转换、发送配置等功能的接口调用主流程示意图。

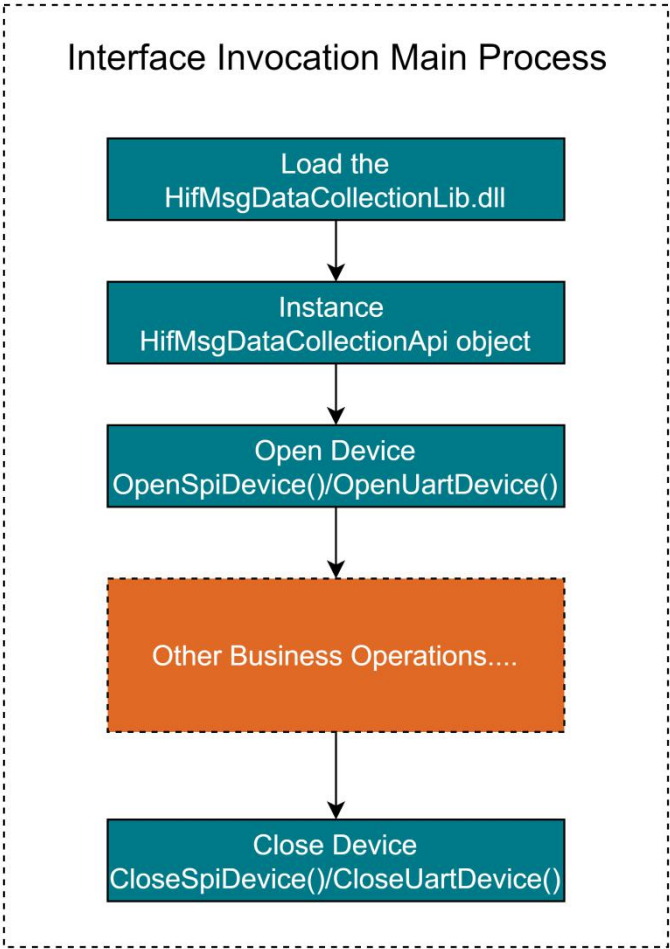


图 3-1 API 接口调用主流程示意图

API 接口调用主流程：

- 1、加载 HifMsgDataCollectionLib.dll 文件；
- 2、实例 HifMsgDataCollectionApi 对象；
- 3、打开设备（OpenUartDevice()/OpenSpiDevice()）；
- 4、根据需求执行相对应的业务流程（该流程可为后续章节 3.2、章节 3.3、章节 3.4 的业务流程）；
- 5、关闭设备（CloseUartDevice()/CloseSpiDevice()）。

注意：若需要多个设备进行通信调用，则需要实例多个 HifMsgDataCollectionApi 对象，不可共用一个 API 接口对象。

3.2 数据采集流程

如下图所示，为通过调用 HifMsgDataCollectionLib.dll 提供的 API 接口实现数据采集、数据转换的流程示意图；开发者可以将数据采集和数据转换分开处理，如此便不会阻塞采数回调事件的执行，避免影响数据采集效率。

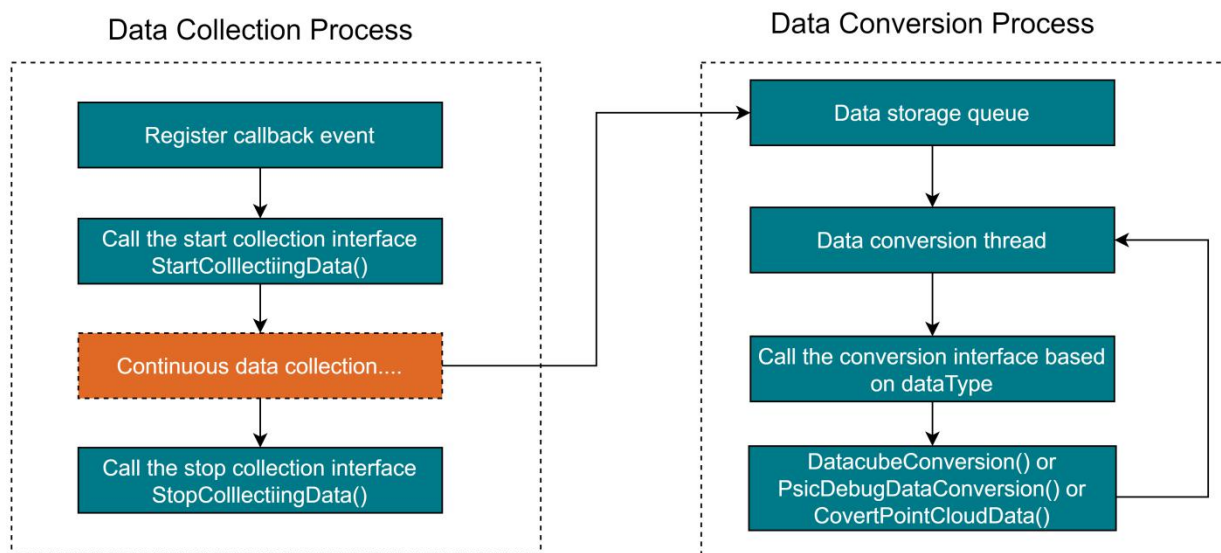


图 3-2 数据采集流程示意图

数据采集流程：

- 1、根据章节 3.1 进行 API 接口对象实例后，在章节 3.1 的第四步执行以下步骤；
- 2、注册数据接收一帧的回调事件，在回调事件中将数据保存至队列中；
- 3、调用启动采数接口（StartCollectingData()）；
- 4、若需要停止采数，调用停止采数接口（StopCollectingData()）；

数据转换流程：

- 1、从队列中取出数据；
- 2、根据 dataType 类型调用相应的数据转换接口（DatacubeConversion()/PsicDebugDataConversion()/CovertPointCloudData()）。

注意：需要先运行固件，在按照上述流程执行操作；

3.3 发送配置文件或发送自定义数据流程

用户若需要发送配置文件或其他数据可以按照下图所示的流程调用相应的 API 接口进行发送。

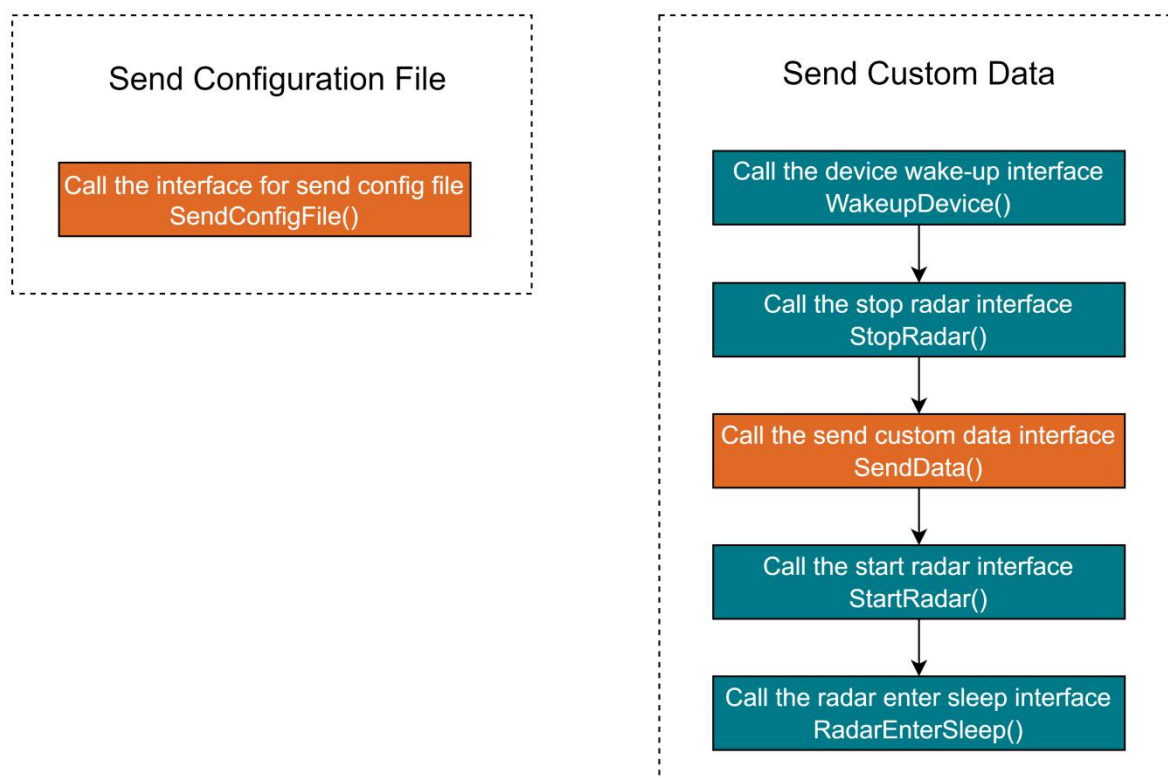


图 3-3 发送配置流程示意图

发送配置文件流程：

- 1、根据章节 3.1 进行 API 接口对象实例后，在章节 3.1 的第四步执行以下步骤；
- 2、调用发送配置文件接口（SendConfigFile(string configFile)）进行发送，无需再调用其他接口。

发送自定义数据流程：

- 1、根据章节 3.1 进行 API 接口对象实例后，在章节 3.1 的第四步执行以下步骤；
- 2、唤醒设备(WakeupDevice)；
- 3、停止雷达(StopRadar)；
- 4、发送数据(SendData)；
- 5、启动雷达(StartRadar)；
- 6、使雷达进入休眠(RadarEnterSleep)；

3.4 数据采集过程中发送配置流程

在数据采集过程若用户需要发送配置数据，可以按照下图所示的流程调用相应的接口进行配置

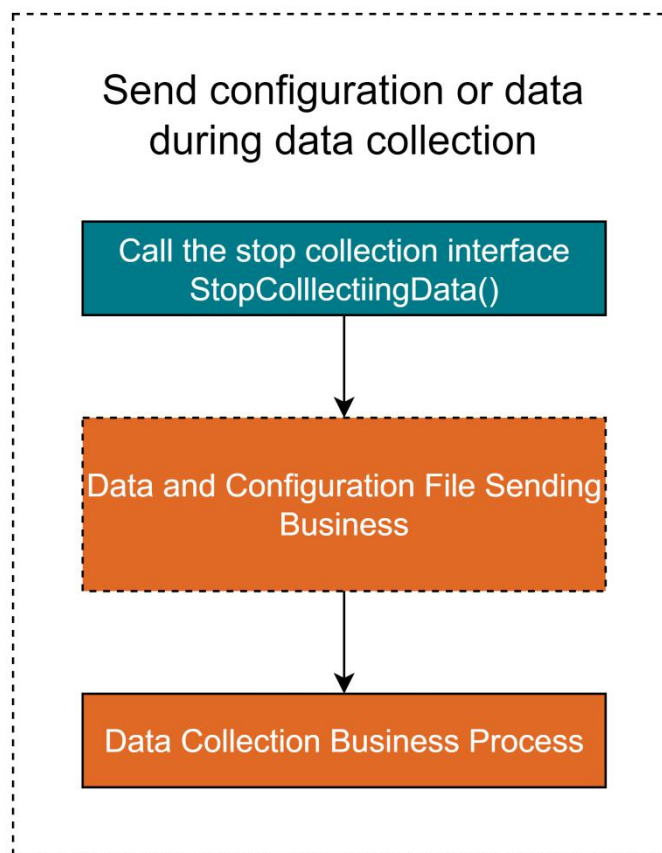


图 3-4 数据采集过程中发送配置的流程示意图

数据采集过程中发送配置的流程为先停止数据采集，再发送配置，然后恢复数据采集：

- 1、根据章节 3.1 进行 API 接口对象实例后，在章节 3.1 的第四步执行以下步骤；
- 2、停止数据采集(StopCollectingData)；
- 3、发送配置，此步根据配置的数据类型参考章节 3.3 执行；
- 4、恢复数据采集，调用 StartCollectingData 或参考章节 3.2 执行；

3.5 数据转换接口关系说明

用户在注册回调事件后所获取的数据需要根据下表的对应关系，调用对应的接口转换方可得到有效数据。

转换接口说明请参考章节 4，回调事件等数据类型请参考章节 5。

注意：dataType 与裸数据一列均为回调事件返回类的属性，需根据不同的 dataType 获取不同格式的裸数据。

表 3-1 不同类型数据对应的转换接口说明表

回调事件	数据类型(dataType)	裸数据	对应的转换接口
public event EventHandler<ReceiveDataEven> callback;	0, psic_debug_data 对应 Hif 消息 ID=0xC6	byte[] payloadData	PsicDebugData PsicDebugDataConversion(byte[] sourceData)
	1, 字符串对应 Hif 消息 ID=0xC7		直接将数据格式化为字符串即可
	2, datacube 对应 Hif 消息 ID=0xC1	uint[] dataCubeRawData	DataCubeData DatacubeConversion(uint[] sourceData, uint frameIndex, uint frameLen, ushort rangeFftPointNum, ushort dopFftPointNum, byte txAntNum, byte rxAntNum)
	3, datacube 对应 Hif 消息 ID=0xC2	byte[] payloadData	DataCubeData DatacubeConversion(byte[] sourceData, uint frameIndex, uint frameLen)
	4, point_cloud 对应 Hif 消息 ID=0xC3	byte[] payloadData	PointCloudData[] ConvertPointCloudData(byte[] payloadData, uint frameIndex, uint frameLen)

4 API 接口说明

4.1 API 接口说明

表 4-1 API 接口说明

接口	参数	说明
string GetLibVersion()	无	获取本 Lib 版本号
bool OpenSpiDevice(uint index, byte freqMode = 0)	index: SPI 设备序号，典型值 0； 注意，一台 PC 只能插入一个 SPI 设备，否则无法识别设备序号对应的 SPI 设备； freqMode: SPI 频率，0=60Mhz、1=30Mhz、2=15MHz、3=7.5MHz、4=3.75MHz、5=1.875MHz、6=937.5KHz、7=468.75KHz； 注意，由于 Datacube 数据量比较大或需要采集数据的帧率比较高，SPI 需用 60MHz；	打开 SPI 设备，与毫米波雷达芯片建立通信渠道；
bool CloseSpiDevice(uint index)	index: SPI 设备序号，典型值 0；	关闭 SPI 设备，断开与毫米波雷达芯片的通信渠道；
bool OpenUartDevice(string com, string baudrate)	com:串口号； baudrate: 波特率；	打开 UART 设备，与毫米波雷达芯片建立通信渠道；
bool CloseUartDevice()	无	关闭 UART 设备，断开与毫米波雷达芯片的通信渠道；
void EnableRetransmission()	无	启用 Datacube 数据错误重传； 默认开启，且 spi 通信方式下起作用；
void DisableRetransmission()	无	禁用 Datacube 数据错误重传
bool StartCollectingData()	无	启动数据采集
bool StopCollectingData()	无	停止数据采集

接口	参数	说明
DataCubeData DatacubeConversion(uint[] sourceData, uint frameIndex, uint frameLen, ushort rangeFftPointNum, ushort dopFftPointNum, byte txAntNum, byte rxAntNum)	此转换接口为裸数据类型 dataType = 2 的转换接口。 sourceData: 需要转换的 Datacube 原始数据, 参数为 uint 类型数组; frameIndex: 表示这是第几帧 Datacube 数据, 在数据采集的回调事件中获取; frameLen: 表示该帧 Datacube 数据的长度, 在数据采集的回调事件中获取; rangeFftPointNum: 距离 FFT 点数, 需要开发者根据实际调试输入; dopFftPointNum: 多普勒 FFT 点数, 需要开发者根据实际调试输入; txAntNum: tx 天线数量, 需要开发者根据实际输入; rxAntNum: rx 天线数量, 需要开发者根据实际输入; 返回类型: DataCubeData, 是一个包含 Datacube 每组天线数据的类对象, 详细类型定义请参考后续章节;	该接口将接收到的 Datacube 数据根据天线进行拆分组合; 开发者通过接口可以拿到每组天线的实部数据和虚部数据;
DataCubeData DatacubeConversion(byte[] sourceData, uint frameIndex, uint frameLen)	此转换接口为裸数据类型 dataType = 3 的转换接口。 sourceData: 需要转换为 Datacube 的 hif payload 数据, 参数为 byte 类型数组; frameIndex: 表示这是第几帧 Datacube 数据, 在数据采集的回调事件中获取; frameLen: 表示该帧 Datacube 数据的长度, 在数据采集的回调事件中获取;	该接口将接收到的数据转换为 Datacube; 开发者通过接口可以拿到每组天线的实部数据和虚部数据;
PsicDebugData PsicDebugDataConversion(byte[] sourceData)	此转换接口为裸数据类型 dataType = 0 的转换接口。 sourceData: 需要转换的原始数据, 参数为 byte 类型数组; 返回类型: PsicDebugData, 是一个包含了数据名字、维度等信息的类对象, 详细类型定义请参考后续章节;	该接口将接收到的类型为 psic_debug_data 的 Hif 消息中的 payload 数据转换为有效数据, 及其相关的信息;

接口	参数	说明
PointCloudData[] ConvertPointCloudData(byte[] payloadData, uint frameIndex, uint frameLen)	此转换接口为裸数据类型 dataType = 4 的转换接口。 payloadData: 需要转换为点云的 hif payload 数据, 参数为 byte 类型数组; frameIndex: 表示这是第几帧点云数据, 在数据采集的回调事件中获取; frameLen: 表示该帧点云数据的长度, 在数据采集的回调事件中获取; 返回类型: PointCloudData[], 是一个 PointcloudData 数组, 每一个数组元素为一种点云数据, 可通过该类的 signalName 属性判断点云类型;	该接口将接收到的类型为 point_cloud 的 Hif 消息中的 payload 数据转换为有效点云数据, 及其相关的信息;
bool WakeupDevice()	无	用于通信交互前, 唤醒 Device;
int StopRadar()	无	停止雷达;
int StartRadar()	无	启动雷达;
int RadarEnterSleep()	无	使雷达进入休眠;
int SendData(byte[] data)	Data: 需要发送的数据	该接口用于发送自定义的数据;
int SendConfigFile(string configFile)	configFile: 需要发送的配置文件	该接口用于发送配置文件;

4.2 错误码说明

表 4-2 错误码说明

错误码	说明
-8	数据或配置文件为空，该错误出现在发送数据时数据为空或发送配置文件时，配置文件为空；
-7	配置失败
-6	启动雷达失败，出现该错误一般都是未唤醒设备导致的；
-5	停止雷达失败，出现该错误一般都是未唤醒设备导致的；
-4	唤醒失败
-3	Ack 超时，未接收到回复的命令；
-2	Ack 错误（checksum error），该错误由通信误码导致的；
-1	未停止采数，该错误是因为在数据采集过程中，未停止采数就发送配置导致的；
0	成功；
1	命令不支持，雷达固件未支持该命令；
5	参数非法（参数越界等），配置的参数越界；
9	雷达设备忙；

5 相关数据类型

5.1 数据采集回调事件类型

HifMsgDataCollectionLib.dll 提供数据接收事件注册接口，以保证在接收到毫米波雷达芯片上报的完整一帧数据时能够及时的将数据传递给上层应用；注册的事件类型如下：

```
/// <summary>
/// 回调事件返回的接收到数据的类
/// </summary>
public event EventHandler<ReceiveDataEven> callback;
```

ReceiveDataEven 类型定义如下：

```
/// <summary>
/// 回调事件返回的接收到数据的类
/// </summary>
public class ReceiveDataEven : EventArgs
{
    /// <summary>
    /// 数据类型： 0,psic_debug_data; 1,字符串; 2, datacube_0xC1;
    /// 3, datacube_0xC2 (新增) ; 4, point_cloud (新增)
    /// </summary>
    public byte dataType { get; set; }

    /// <summary>
    /// 表示第几帧数据，仅dataType为2、3、4时数据有效
    /// </summary>
    public UInt32 frameIndex { get; set; }

    /// <summary>
    /// 表示该帧数据长度，仅dataType为2、3、4时数据有效
    /// </summary>
    public UInt32 frameLen { get; set; }

    /// <summary>
    /// 时间戳，表示接收到该帧数据的时间
    /// </summary>
    public string timeStamp { get; set; }

    /// <summary>
    /// 点云裸数据/字符串，hif消息的payload,后续需要转换为有效数据
    /// </summary>
    [Obsolete("此属性已废弃，后期会彻底废弃，请使用payloadData替代")]
    public byte[] pointCloudRawDataOrStringData { get; set; }
```

```
/// <summary>
/// datacube数据
/// </summary>
public UInt32[] datacubeRawData { get; set; }

/// <summary>
/// hif消息的payload，后续需要转换为有效数据
/// </summary>
public byte[] payloadData{ get; set; }
}
```

注：在获取数据时需要根据数据类型 (dataType) 读取数据；当 **dataType = 0 、1、3、4** 时，读取 payloadData 数组即可获取相应的裸数据；当 **dataType = 2** 时，读取 datacubeRawData 数组即可获取消息 id 0xC1 上报的 Datacube 数据；

5.1.1 UInt32[] datacubeRawData 数据格式

UInt32[] datacubeRawData 数据格式如下图所示，开发者可根据实际距离 FFT 点数和多普勒 FFT 点数以及 tx、rx 天线数量进行每组 tx、rx 天线数据的拆分获取，也可以通过 DatacubeConversion API 接口获取；

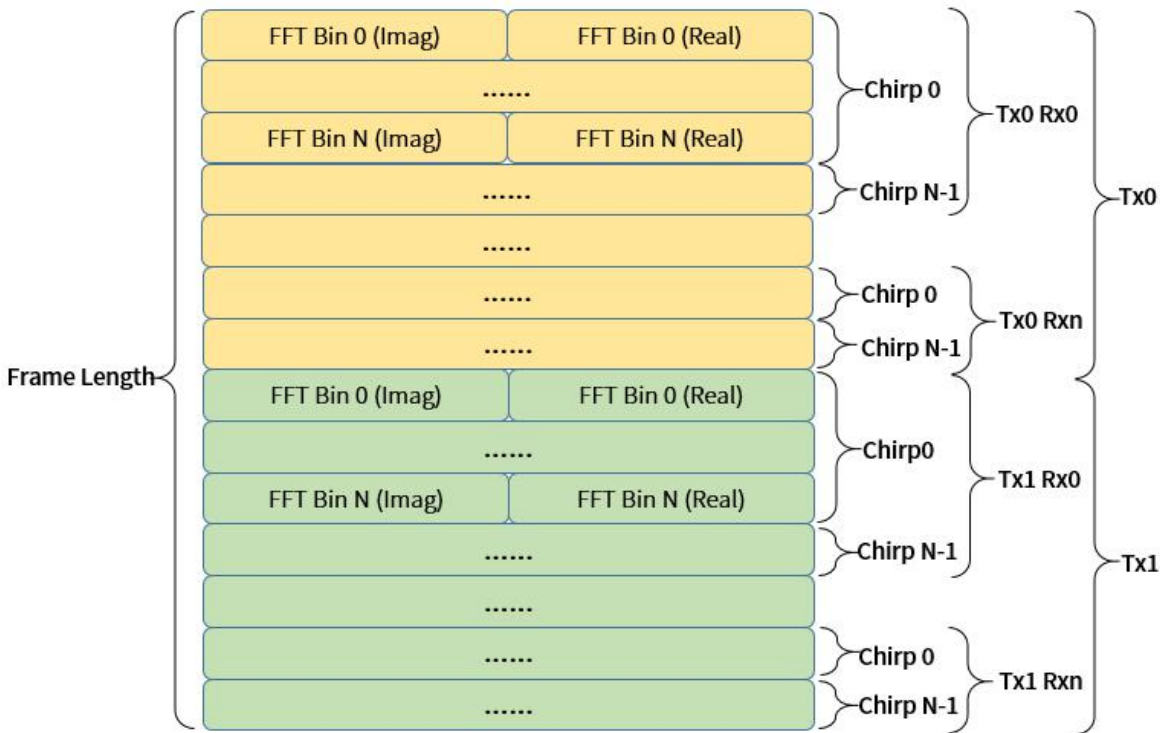


图 5-1 UInt32[] datacubeRawData 数据格式

5.1.2 当 dataType = 0 or 1 时，byte[] payloadData 数据格式

byte[] pointCloudRawDataOrStringData 的数据格式如下图所示，开发者可根据定义解析获取毫米波雷达芯片上报数据的真实值以及相关信息

当 dataType = 0 时，数据格式如下：

Hif Payload Data(0~4095 bytes), Msg_Id = 0xC6

Payload Header			Payload Data	
Dim	DataFormat	NumofOneDim	Payload Name	Payload Data
8bits	16bits	16bits	字符串，以'\0'结尾	上报的数据，格式由dataformat控制

图 5-2 dataType = 0 时，byte[] pointCloudRawDataOrStringData 数据格式

表 5-1 dataType = 0 时，byte[] pointCloudRawDataOrStringData 参数定义

参数	长度	说明
Dim	8 bits	数据维度（N: N 维数据），0 和 1 都表示 1D 数据
DataFormat	16 bits	bit0~bit1 表示数据类型 0: byte; 1: short; 2: word; 3: 浮点类型;
		bit2 表示数据是否有符号 数据类型=0/1/2 时: 0: 无符号数; 1: 有符号数; 数据类型=3 时: 0: 单精度浮点数; 1: 双精度浮点数;
		Bit3~bit7 表示定点化位数: 上报数据的真实值 = 总线值 / pow(2, 定点化位数)
		bit8 表示 payload data 的排列方式: 0: 数据长度优先; x1, x2, . . . , xN, y1, y2, . . . , yN 1: dim 优先; x1, y1, x2, y2, . . . , xN, yN
NumOfOneDim	16 bits	单维数据个数
Payload Name	以实际长度为准	上报的数据名字
Payload Data	以实际长度为准	上报的数据，格式由 dataformat 控制; 总数据长度 = Dim * NumOfOneDim; 注意：当上报的 dim=0 时，取 dim = 1 计算

当 `dataType = 1` 时，上报的数据为字符串，开发者将整个数组进行字符串格式化即可，数据格式如下：

Hif Payload Data(0~4095 bytes), Msg_Id = 0xC7	
Payload Data	
字符串，以'\0'结尾	

图 5-3 `dataType = 1` 时，`byte[] pointCloudRawDataOrStringData` 数据格式

5.2 数据转换返回 `PsicDebugData` 类型

`PsicDebugData` 类型定义如下：

```
/// <summary>
/// psic debug数据
/// </summary>
public class PsicDebugData
{
    /// <summary>
    /// 维度，表示接收到信号数据是几维的，0和1都表示只有1D数据（只有y）
    /// 目前最大支持6D数据接收
    /// </summary>
    public byte dim;

    /// <summary>
    /// 数据名字
    /// </summary>
    public string signalName;

    /// <summary>
    /// 数据个数
    /// </summary>
    public UInt32 pointNum;

    /// <summary>
    /// 有效数据，根据维度dim取相应的数据；若dim=0或1，则只有一维数据y
    /// </summary>
    public double[] x;
    public double[] y;
    public double[] z;
    public double[] w;
    public double[] u;
    public double[] v;
}
```

开发者在回调事件中获取到的 `dataType=0` 的数据后，可以通过 `PsicDebugDataConversion` 接口解析得到真正上报的数据以及其数据名字；`dim` 值决定 `x`、`y`、`z`、`w`、`u`、`v` 数组中是否有数据，没有则为 `null`。

如 `dim=1`、`signalName="debug_data"`、`pointNum = 5` 表示上报的数据有 5 个，并存放在 `y` 数组中。

注意：`dim=0` 或 `1` 时，表示只有一维数据，只有 `y`。

5.3 数据转换返回 PointCloudData 类型

PointCloudData 类型定义如下：

```
/// <summary>
/// 点云数据
/// </summary>
public class PointCloudData
{
    /// <summary>
    /// 维度，表示接收到信号数据是几维的，0和1都表示只有1D数据（只有y）
    /// 目前最大支持6D数据接收
    /// </summary>
    public byte dim;

    /// <summary>
    /// 点云名字
    /// </summary>
    public string signalName;

    /// <summary>
    /// 点云个数
    /// </summary>
    public UInt32 pointNum;

    /// <summary>
    /// 有效数据，根据维度dim取相应的数据；若dim=0或1，则只有一维数据y
    /// dim=5 时，x、y、z 的单位是米(m)，w 是速度(单位： m/s)，u 是 snr(单位： db)
    /// </summary>
    public double[] x;
    public double[] y;
    public double[] z;
    public double[] w;
    public double[] u;
    public double[] v;
}
```

开发者在回调事件中获取到的 `dataType=4` 数据后，则调用 `ConvertPointCloudData` 接口解析得到有效的点云数据。dim 值决定 x、y、z、w、u、v 数组中是否有数据，没有则为 null。

如 dim=3、signalName="motion_point_cloud"、pointNum = 5 表示上报的数据为移动点云且有 5 个点，数据为每个点的 x、y、z 坐标。

注意：dim=0 或 1 时，表示只有一维数据，只有 y。dim=5 时，x、y、z 的单位是米(m)，w 是速度(单位：m/s)，u 是 snr(单位：db)。

5.4 数据转换返回 DataCubeData 类型

DataCubeData 类型定义如下：

```
/// <summary>
/// 返回转换好的datacube数据
/// </summary>
public class DataCubeData
{
    /// <summary>
    /// 表示这是第几帧的datacube
    /// </summary>
    public UInt32 frameIndex;

    /// <summary>
    /// datacube的数据长度
    /// </summary>
    public UInt32 frameLen;

    /// <summary>
    /// 距离fft点数
    /// </summary>
    public UInt16 rangeFftPointNum;

    /// <summary>
    /// 多普勒fft点数
    /// </summary>
    public UInt16 dopFftPointNum;

    /// <summary>
    /// tx天线数量
    /// </summary>
    public byte txAntNum;

    /// <summary>
    /// rx天线数量
    /// </summary>
    public byte rxAntNum;

    /// <summary>
    /// 每组天线的数据，一组天线数据为一个AntennaData对象
    /// </summary>
    public AntennaData[] antennaData;
}
```

开发者在回调事件中获取到的 `dataType=2 or 3` 的数据后，可通过 `DatacubeConversion` 接口解析得到每一组 txrx 天线的的数据；每组数据用一个 `AntennaData` 类存储，其包含了每组数据天线 id 以及实部数据和虚部数据；`AntennaData` 定义如下：

```
/// <summary>
/// 天线的数据
/// </summary>
public class AntennaData
{
    /// <summary>
```

```
/// tx天线id
/// </summary>
public byte txAntId;

/// <summary>
/// rx天线id
/// </summary>
public byte rxAntId;

/// <summary>
/// 实部数据
/// </summary>
public double[] real;

/// <summary>
/// 虚部数据
/// </summary>
public double[] imag;
}
```

6 使用示例 (Python)

6.1 主流程

本示例演示了 Python 如何通过 pythonnet 调用 hifMsgDataCollectionLib，实现雷达上报数据的接收与解析流程。

执行主流程：

1. 通过封装好的导入 dll 类 DllLoader 加载 HifMsgDataCollectionLib，获取 HifMsgDataCollectionLib 命名空间；
2. 从 HifMsgDataCollectionLib 命名空间导入 HifMsgDataCollectionApi 类和 ReceiveDataEven 数据结构体对象；
3. 实例化 HifMsgDataCollectionLib 的 HifMsgDataCollectionApi 类，用于调用 API 接口；
4. 实例化 Python 的数据接收与处理类 DataCollection，用于接收和解析上报数据；
5. 调用 DataCollection 对象的 OpenSpiDevice() 方法，打开 SPI 设备；
6. 调用 DataCollection 对象的 StartCollectingData() 方法，启动数据采集；
7. 通过 while 循环模拟持续采集过程，实时接收雷达上报的数据。

```
if __name__ == "__main__":
    data_collection = None
    try:
        DllLoader(DLL_FILE)
        from HifMsgDataCollectionLib import HifMsgDataCollectionApi

        api = HifMsgDataCollectionApi()
        data_collection = DataCollection(api)
        data_collection.GetLibVersion()

        if not open_device(data_collection):
            sys.exit(1)

        data_collection.StopCollectingData()
        send_config_file(data_collection)

        data_collection.StartCollectingData()
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        print("用户停止程序")
    except Exception as e:
        print(f"程序运行出错: {str(e)}")
```

6.2 Python 环境的准备

环境要求：

1. 安装 Python3.8 或更高版本；
2. 安装 pythonnet: pip install pythonnet;
3. 文件准备：将 HifMsgDataCollectionLib.dll 和 DataCollection_demo.py 放在同级目录；

脚本头部导入：

```
import os
import sys
import time
import queue
import clr
import importlib
import threading
from System import Array, Byte
```

6.3 DllLoader 类

表 6-1 DllLoader 类说明

分类	说明
类	加载 HifMsgDataCollectionLib 库
功能	加载 HifMsgDataCollectionLib，获取 HifMsgDataCollectionLib 的命名空间以及接口对象
参数	DLL 文件绝对路径

```
class DllLoader:
    """加载 DLL 并获取命名空间。"""

    def __init__(self, dll_path: str):
        if not os.path.exists(dll_path):
            raise FileNotFoundError(f"DLL 文件不存在: {dll_path}")

        self.dll_path = os.path.abspath(dll_path)
        dll_dir = os.path.dirname(self.dll_path)
        dll_name = os.path.splitext(os.path.basename(self.dll_path))[0]

        if dll_dir not in sys.path:
            sys.path.append(dll_dir)

        clr.AddReference(dll_name)
        self.namespace = importlib.import_module(dll_name)
```

6.4 DataCollection 类

表 6-2 DataCollection 类说明

分类	说明
类	数据接收和处理类
功能	接收上报的数据，调用接口解析，并将结果进行打印输出
参数	HifMsgDataCollectionApi 类对象

```
class DataCollection:
    """数据采集和接口封装。"""

    DATA_TYPE_NAME = {
        0: "psic_debug",
        1: "字符串",
        2: "datacube_0xC1",
        3: "datacube_0xC2",
        4: "point_cloud",
    }

    def __init__(self, data_collection_api):
        self.api = data_collection_api
        self.api.callback += self.data_callback
        self.data_queue = queue.Queue(maxsize=1000)
```

6.4.1 初始化函数

函数参数为 HifMsgDataCollectionApi 类对象，调用该对象的 callback 绑定 python 回调函数，并设定数据缓存队列大小，同时包含了要修改的解析数据的天线配置。

```
def __init__(self, data_collection_api):
    self.api = data_collection_api
    self.api.callback += self.data_callback
    self.data_queue = queue.Queue(maxsize=1000)
    self.running = True

    # DataCube 解析参数，按实际雷达配置修改
    self.rangeFftPointNum = 256
    self.dopFftPointNum = 32
    self.txAntNum = 2
    self.rxAntNum = 4

    # 后台线程处理回调数据
    self.worker = threading.Thread(target=self.process_data, daemon=True)
    self.worker.start()
```

6.4.2 Python 回调函数

函数参数为触发事件的对象和数据结构体对象, 当 HifMsgDataCollectionApi 对象触发回调事件时, Python 接收 HifMsgDataCollectionApi 返回的数据类对象, 并塞入数据缓存队列。

```
def data_callback(self, sender, receive_data):
    """接收回调中只入队, 避免阻塞 DLL。"""
    try:
        self.data_queue.put_nowait(receive_data)
    except queue.Full:
        print("队列已满, 丢弃数据")
```

6.4.3 数据转换线程函数

后台数据转换线程函数, 将数据结构体对象从队列取出, 并传递给数据解析函数进行解析, 将处理的结果打印输出。

```
def process_data(self):
    """后台解析数据。"""
    while self.running:
        try:
            data = self.data_queue.get(timeout=1)
            self.parse_data(data)
            self.data_queue.task_done()
        except queue.Empty:
            continue
```

```
def parse_data(self, receive_data):
    """按数据类型解析一帧数据。"""
    print("*" * 20)
    data_type = int(receive_data.dataType)
    type_name = self.DATA_TYPE_NAME.get(data_type, f"未知类型({data_type})")
    print(f"时间戳: {receive_data.timeStamp}")
    print(f"数据类型: {type_name}")

    if data_type == 0:
        self._parse_psic_debug(receive_data)
    elif data_type == 1:
        self._parse_string_data(receive_data)
    elif data_type == 2:
        self._parse_datacube_c1(receive_data)
    elif data_type == 3:
        self._parse_datacube_c2(receive_data)
    elif data_type == 4:
        self._parse_point_cloud(receive_data)
    else:
        payload = self._get_payload_bytes(receive_data)
        raw_len = len(payload) if payload is not None else 0
```

```
print(f"未处理的数据类型，rawLen={raw_len}")
```

6.4.4 下发配置文件示例

当需要通过配置文件完成参数下发时，可在设备打开后调用 `SendConfigFile(configFile)` 接口。

Python 示例中可通过设置 `CONFIG_FILE` 并调用 `send_config_file(data_collection)` 完成配置文件下发。

注意：调用 `SendConfigFile(configFile)` 接口前需调用 `StopCollectingData()` 接口停止采数；

```
def send_config_file(data_collection):
    """按配置文件一键下发。"""
    if not CONFIG_FILE:
        return
    data_collection.StopCollectingData()
    data_collection.SendConfigFile(CONFIG_FILE)
```

6.4.5 自定义数据发送示例

当需要发送自定义数据时，可按 `StopCollectingData()`、`WakeupDevice()`、`StopRadar()`、`SendData()`、`StartRadar()` 的顺序调用；如业务需要，可在最后调用 `RadarEnterSleep()`。

```
def send_config_by_steps(data_collection, config_data):
    """按原子接口下发配置数据示例。"""
    data_collection.StopCollectingData()
    data_collection.WakeupDevice()
    data_collection.StopRadar()
    data_collection.SendData(config_data)
    data_collection.StartRadar()
```

6.4.6 数据解析输出示例

示例脚本会根据 `dataType` 对上报数据进行分类解析，并在后台线程中输出简略结果。为避免日志过多，示例仅打印时间戳、数据类型、点数、帧号、天线数量，以及首个点或首根天线的摘要信息。

```
def parse_data(self, receive_data):
    """按数据类型解析一帧数据。"""
    print("*" * 20)
    data_type = int(receive_data.dataType)
    type_name = self.DATA_TYPE_NAME.get(data_type, f"未知类型({data_type})")
    print(f"时间戳：{receive_data.timeStamp}")
    print(f"数据类型：{type_name}")

    if data_type == 0:
        self._parse_psic_debug(receive_data)
    elif data_type == 1:
        self._parse_string_data(receive_data)
    elif data_type == 2:
        self._parse_datacube_c1(receive_data)
    elif data_type == 3:
        self._parse_datacube_c2(receive_data)
```

```
elif data_type == 4:  
    self._parse_point_cloud(receive_data)
```

若需要完整数据，可在此基础上自行保存原始数组或扩展打印内容。

6.5 数据解析说明

回调函数接收到数据后，通过 `parse_data` 方法根据 `receive_data.dataType` 的值分发到对应的解析函数。各数据类型的解析方式如下：

6.5.1 dataType = 0: psic_debug 解析

当 `dataType = 0` 时，表示接收到 `psic_debug` 类型的点数据。从 `receive_data.payloadData` 获取原始字节数据后，调用 `PsicDebugDataConversion` 接口进行解析，返回 `PointCloudData` 对象。

`PointCloudData` 关键字段说明：

- `signalName`：信号名称（如 "debug_data"）
- `dim`：数据维度（决定哪些坐标轴有效）
- `pointNum`：点数量
- `x[], y[], z[], w[], u[], v[]`：各维度坐标数组

```
def _parse_psic_debug(self, receive_data):  
    """解析 debug 点数据。"""  
    payload = self._get_payload_bytes(receive_data)  
    if payload is None:  
        print("psic_debug 数据为空")  
        return  
  
    if self._has_api_method("PsicDebugDataConversion"):  
        point_cloud = self.api.PsicDebugDataConversion(payload)  
    else:  
        point_cloud = self.api.PointCloudConversion(payload)  
    self._print_point_cloud_summary(point_cloud)
```

6.5.2 dataType = 1: 字符串数据解析

当 `dataType = 1` 时，上报数据为字符串格式。从 `receive_data.payloadData` 获取原始字节后，按 UTF-8 或 GBK 编码解码即可得到文本内容。

```
def _parse_string_data(self, receive_data):  
    """解析字符串数据。"""  
    payload = self._get_payload_bytes(receive_data)  
    text = self._decode_text(payload)  
    print(f"长度={len(text)}, 内容={self._preview_text(text)}")
```

6.5.3 dataType = 2: datacube_0xC1 解析

当 dataType = 2 时, 表示接收到消息 ID 为 0xC1 的 Datacube 数据。从 receive_data.datacubeRawData 获取原始数据后, 调用 DatacubeConversion 接口解析, 需传入雷达配置参数。

DatacubeConversion 参数说明:

参数名	类型	说明
datacubeRawData	UInt32[];	回调事件中的原始 datacube 数据
frameIndex	UInt32	帧索引 (回调事件中获取)
frameLen	UInt32	帧长度 (回调事件中获取)
rangeFftPointNum	int	距离 FFT 点数 (按雷达配置填写)
dopFftPointNum	int	多普勒 FFT 点数 (按雷达配置填写)
txAntNum	int	发射天线数量

返回 DataCubeData 对象, 其中 antennaData 数组包含每根天线的数据, 每个 AntennaData 包含 txAntId、rxAntId、real[] (实部) 和 imag[] (虚部)。

```
def _parse_datacube_c1(self, receive_data):
    """解析 0xC1 datacube。"""
    data_cube = self.api.DatacubeConversion(
        receive_data.datacubeRawData,
        receive_data.frameIndex,
        receive_data.frameLen,
        self.rangeFftPointNum,
        self.dopFftPointNum,
        self.txAntNum,
        self.rxAntNum,
    )
    self._print_datacube_summary(data_cube)
```

6.5.4 dataType = 3: datacube_0xC2 解析

当 dataType = 3 时, 表示接收到消息 ID 为 0xC2 的 Datacube 数据。从 receive_data.payloadData 获取原始数据后, 调用 DatacubeConversion 接口解析 (仅需 3 个参数, FFT 参数已包含在数据中)。返回类型与 0xC1 相同。

```
def _parse_datacube_c2(self, receive_data):
    """解析 0xC2 datacube。"""
    payload = self._get_payload_bytes(receive_data)
    if payload is None:
        print("datacube_0xC2 数据为空")
        return
    if self._has_api_method("DatacubeConversion"):
        try:
            data_cube = self.api.DatacubeConversion(
```

```

        payload,
        receive_data.frameIndex,
        receive_data.frameLen,
    )
    self._print_datacube_summary(data_cube)
    return

```

6.5.5 dataType = 4: point_cloud 解析

当 dataType = 4 时，表示接收到点云数据。从 receive_data.payloadData 获取原始数据后，调用 ConvertPointCloudData 接口解析，返回 PointCloudData 列表。每个 PointCloudData 的字段含义与 dataType=0 相同。

dim 与坐标轴对应关系：

- dim=1: 仅 y 有效
- dim=3: x, y, z 有效（单位：m）
- dim=5: x, y, z (m) , w (速度, m/s) , u (SNR, dB) 有效

```

def _parse_point_cloud(self, receive_data):
    """解析新增点云数据。"""
    payload = self._get_payload_bytes(receive_data)
    if payload is None:
        print("point_cloud 数据为空")
        return

    if not self._has_api_method("ConvertPointCloudData"):
        print(f"当前 DLL 不支持 point_cloud 解析，rawLen={len(payload)}")
        return

    point_cloud_list = self.api.ConvertPointCloudData(
        payload,
        receive_data.frameIndex,
        receive_data.frameLen,
    )
    if point_cloud_list is None:
        print("point_cloud 解析结果为空")
        return

    print(f"pointCloudCount={len(point_cloud_list)}")
    for point_cloud in point_cloud_list:
        self._print_point_cloud_summary(point_cloud)

```

7 常见问题

7.1 为什么打开串口失败

表 7-1 串口打开异常

可能原因	措施
1、当前 windows 操作系统未安装该设备的串口驱动；	根据 windows 操作系统提示信息安装相应的串口设备驱动。
2、当前串口被占用；	检查是否有其他设备占用当前串口

7.2 为什么打开 SPI 设备失败

表 7-2 SPI 打开异常

可能原因	措施
1、当前 windows 操作系统未安装 spi 转 usb 的转接芯片驱动；	安装 CH347T 转接芯片驱动；
2、开发板的通信方式拨码未拨至 UART/IIC/SPI 方式位置；	将开发板的通信方式拨码开关未拨至 UART/IIC/SPI 方式位置，并重新上下电；

7.3 为什么没有接收到数据或打印数据不完整

表 7-3 没接收到数据

可能原因	措施
若上报的 datacube 数据，可能帧率太高导致数据传输不完整被丢弃；	开控制台： 观察打印的 log 信息，是否出 “Datacube/Point cloud data transmission incomplete,please reduce frame rate!” 等信息，若出现则需要降低帧率； 观察是否有打印 “checksum error” 等信息，若出现则需检查雷达传感器数据是否有正确传输；
通信拨码开关未正确拨至 “ON”	将开发板上第 9 号 IRQ 拨码开关拨至 “ON”

可能原因	措施
若是 uart 方式，可能波特率不正确	输入正确的波特率

7.4 发送配置异常排查方向

表 7-4 配置异常排查方向

可能原因	措施
1、返回错误码“-1”，数据采集过程中发送配置前未停止数据采集；	先停止数据采集，再发送配置，最后启动数据采集；
2、返回错误码“-3, -4, -5”，雷达传感器可能未正确运行雷达固件；	重新下载对应系列的雷达固件，并在下载完成后重新上电运行；
3、返回错误码“-8”，表示需要发送的配置数据为空，可能是配置文件为空或格式不正确，自定义配置的数据为空；	1、检查配置文件是否为空，格式是否正确(注：不要填写任何注释)； 2、检查自定义的配置数据是否为空；
4、返回错误码“1”，发送的自定义数据的 hif 消息 id 在雷达固件中未定义，或未下载雷达固件；	1、若是使用本司相应的 sdk 自行开发的固件，则需定义该消息 id； 2、检查是否有下载雷达固件；
5、返回错误码“5”，发送的配置数据中存在非法参数，如越界等；	检查配置文件是否有手动修改的痕迹，或自定义的配置的每个数据是否在合理范围内；

免责声明和著作权声明

本司在本文档中提供的信息力求准确，但并不保证内容完全无误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或不遵守本文档使用说明而进行的任何不当使用行为（包括但不限于如超压，超频，超温使用）所造成的损失，本司概不负责。

本文中的内容，包括但不限于供参考的 URL 地址，由于产品版本升级或其他原因，与实际内容存在差异，如有变更，恕不另行通知。文档“按现状”提供，不负任何担保责任，包括对适销性、适用于特定用途或非侵权性的任何担保，和任何提案、规格或样品在他处提到的任何担保。

本文档不负任何责任，包括使用本文档内信息产生的侵犯任何专利权行为的责任。本文档在此未以禁止反言或其他方式授予任何知识产权使用许可，不管是明示许可还是暗示许可。

版权所有©2026 珠海正和微芯科技有限公司。保留所有权利。